Выполнил: Василенко И.А

Группа: ПИа-090303-о24

ЛАБОРАТОРНАЯ РАБОТА №3

Дисциплина: Основы программирования на Python

Тема:

Объектно-ориентированное программирование в Python

Вариант 2: Сотрудники компании

1. Цель работы

Получить практические навыки разработки программ с использованием объектно-ориентированного подхода. В процессе выполнения лабораторной работы освоить понятия класса, объекта, инкапсуляции, наследования, полиморфизма и композиции объектов в языке Python.

2. Постановка задачи

Разработать консольное приложение, моделирующее объекты предметной области «Сотрудники компании». Данные должны быть представлены объектами классов. Программа должна содержать базовый класс Employee, производные классы Developer (разработчик) и Manager (менеджер), а также класс-контейнер Company для демонстрации композиции. Необходимо реализовать инкапсуляцию данных, полиморфизм через общий метод расчета показателей и консольное меню.

3. Краткие теоретические сведения

1) Класс — чертеж или описание структуры и поведения однотипных объектов.

2) Объект (экземпляр) — конкретная сущность, созданная на основе класса и обладающая собственным состоянием.

3) Инкапсуляция — сокрытие внутренних данных объекта и ограничение прямого доступа к ним. В Python реализуется с помощью соглашения об именовании (символ подчеркивания _) и встроенных свойств (@property).

4) Наследование — механизм создания нового класса на основе уже существующего. Для вызова конструктора родительского класса применяется super().

5) Полиморфизм — возможность работы с объектами разных классов через единый интерфейс (одноименные методы с разным поведением).

6) Композиция — отношение типа «HAS-A» (включение), при котором один объект является частью другого и управляет его списком.

4. Текст программы (Исходный код)

```
class Employee:
    def __init__(self, name, age, zp):
        self.name = name
        self.age = age
        self.zp = zp

    @property #Инкапсуляция
    def zp(self):
        return self._zp

    @zp.setter #Инкапсуляция
    def zp(self, value):
        if not isinstance(value, (int, float)):
            raise ValueError("Введите число, а не слово!")
        if value < 0:
            raise ValueError("Зарплата не может быть отрицательной!")
        self._zp = value

class Developer(Employee):
    def __init__(self, name, age, zp, lang, projects):
        super().__init__(name, age, zp) #Наследование
        self.lang = lang
        self.projects = projects

    def calculated(self):
        return self.projects * 10 #Полиморфизм - общий метод

class Manager(Employee):
    def __init__(self, name, age, zp, subord, budget):
        super().__init__(name, age, zp) #Наследование
        self.subord = subord
        self.budget = budget

    def calculated(self):
        return self.subord * 5 #Полиморфизм - общий метод

class Company: #Композиция - содержит в себе объекты people
    def __init__(self, title):
        self.title = title
        self.people = []

    def add_person(self, emp):
```

```
        self.people.append(emp)

def add_devep():
    name = input("Введите имя: ")
    age = int(input("Введите возраст: "))
    zp = float(input("Введите зарплату: "))
    lang = input("Введите язык программирования: ")
    projects = int(input("Введите количество проектов: "))
    new_dev = Developer(name, age, zp, lang, projects)
    my_company.add_person(new_dev)
    return print(f"Разработчик {name} успешно добавлен в компанию!")

def add_mang():
    name = input("Введите имя: ")
    age = int(input("Введите возраст: "))
    zp = float(input("Введите зарплату: "))
    subord = int(input("Введите количество подчинённых: "))
    budget = float(input("Введите бюджет проектов: "))
    new_mang = Manager(name, age, zp, subord, budget)
    my_company.add_person(new_mang)
    return print(f"Менеджер {name} успешно добавлен в компанию!")

def empp():
    for emp in my_company.people:
        perf = emp.calculated()
        print(f"Имя: {emp.name} | Возраст: {emp.age} | Зарплата: {emp.zp} руб. |
Эффективность: {perf}")


my_company = Company("Медоёж")

while True:
    print("\n----МЕНЮ----")
    print("1. Создать сотрудника")
    print("2. Показать всех сотрудников")
    print("0. Выход")

    vibe = input()
    if vibe == "1":
        print("\nКого создать? 1 - Разработчик, 2 - Менеджер")
        who = input("Ваш выбор: ")

        try:
            if who == "1":
                add_devep()
            elif who =="2":
                add_mang()
        # Если класс выкинул ошибку, мы её ловим здесь и не даем программе упасть
        except ValueError as error:
            print(f"\n[ОШИБКА]: {error}")
            print("Сотрудник не был создан. Попробуйте ещё раз.")

    elif vibe =="2":
        if not my_company.people:
            print("В компании пока нет сотрудников.")
        else:
            print(f"\n Сотрудники компании: {my_company.title}")
            empp()
    elif vibe == "0":
        print("Выход из программы")
```

```
        break
    else:
        print("Неверный пункт программы!")
```

5. Результаты тестирования

Согласно разделу 12 требований лабораторной работы, были проведены следующие тесты:

1) Тест 1: Через консольное меню успешно созданы объекты классов Developer и Manager (Рисунок 1.1 и 1.2).

```
Кого создать? 1 - Разработчик, 2 - Менеджер
Ваш выбор: 1
Введите имя: Игорь
Введите возраст: 19
Введите зарплату: 95000
Введите язык программирования: python
Введите количество проектов: 67
Разработчик Игорь успешно добавлен в компанию!

----МЕНЮ----
1. Создать сотрудника
```

Рисунок 1.1 - Создание объектов каждого типа

```
Кого создать? 1 - Разработчик, 2 - Менеджер
Ваш выбор: 2
Введите имя: Игорь
Введите возраст: 19
Введите зарплату: 150000
Введите количество подчинённых: 200
Введите бюджет проектов: 200000
Менеджер Игорь успешно добавлен в компанию!

----МЕНЮ----
1. Создать сотрудника
```

Рисунок 1.2 - Создание объектов каждого типа

2) Тест 2 и Тест 3: Вызов метода calculated() производит верные математические расчеты для каждого подкласса (умножение на 10 для разработчиков, на 5 для менеджеров).

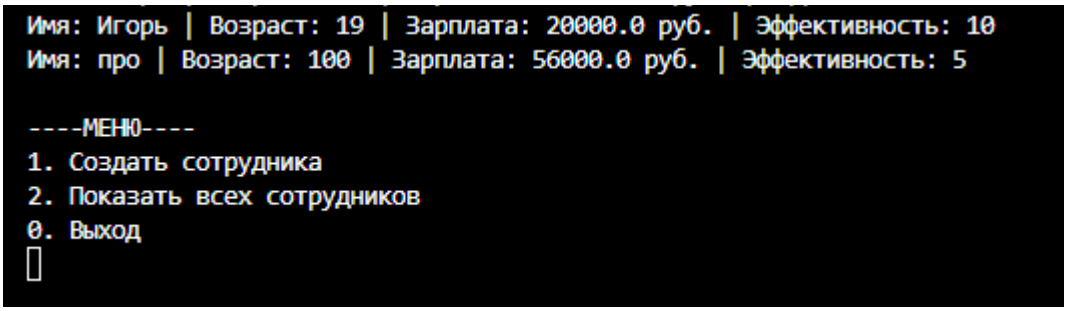


Рисунок 1.3 - Выполнение общих и переопределенных методов

3) Тест 4: Данные сотрудников успешно принимаются через консольный ввод и сохраняются в свойства объектов.

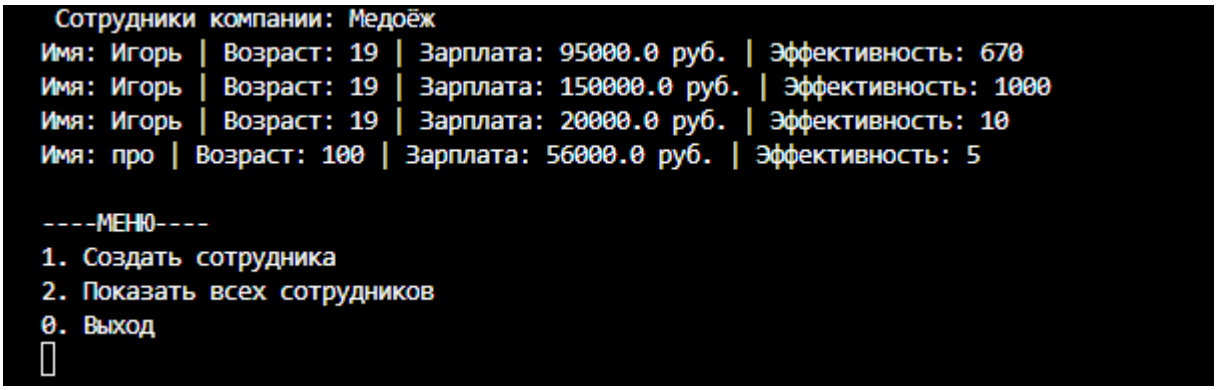


Рисунок 1.4 - Изменение состояния объекта

4) Тест 5: При попытке указать отрицательную зарплату (например, -5000), программа выбрасывает исключение ValueError с соответствующим текстом, блокируя неверные данные.

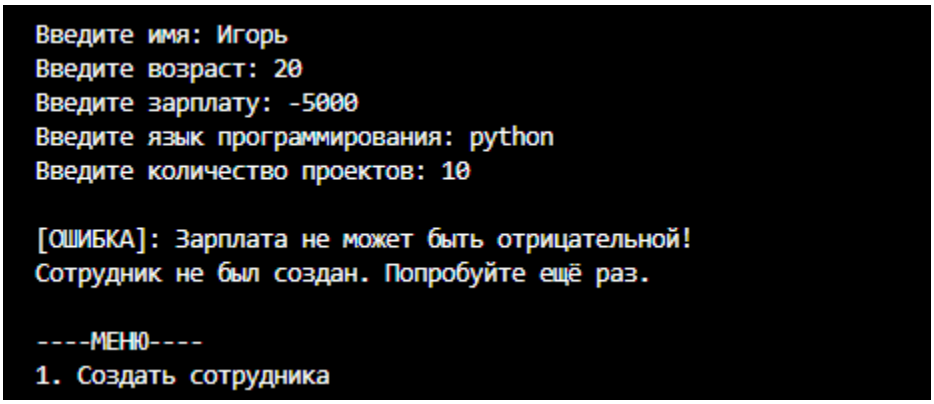


Рисунок 1.5 - Обработка некорректного значения свойства

6. Выводы

В ходе выполнения лабораторной работы были успешно освоены ключевые концепции объектно-ориентированного программирования на языке Python. Была построена гибкая архитектура приложения с использованием базового класса и его наследников. Реализована надежная защита данных (инкапсуляция) с помощью механизма свойств, а также продемонстрированы преимущества полиморфизма и композиции при управлении коллекцией разнородных объектов.